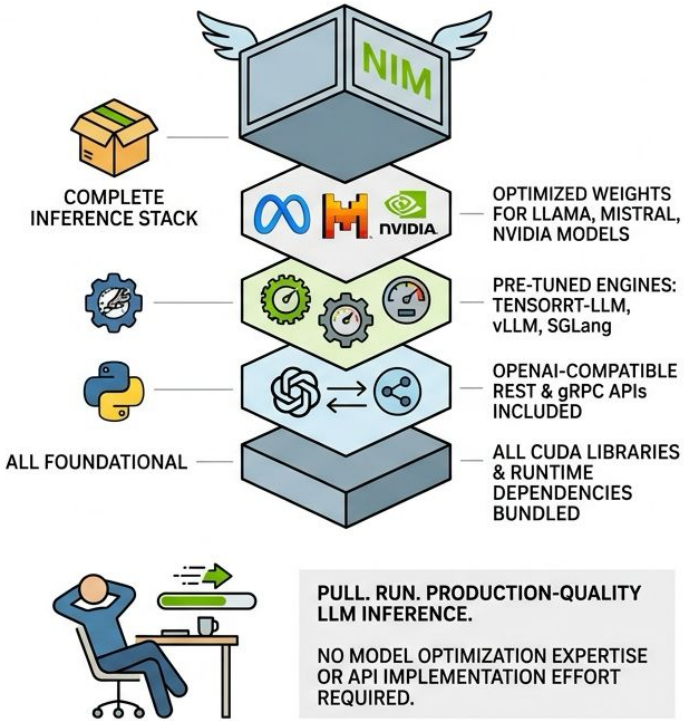
U.S. General Services Administration

Chapter 4.5: NVIDIA NIM and Triton Inference Server

US General Service Administration

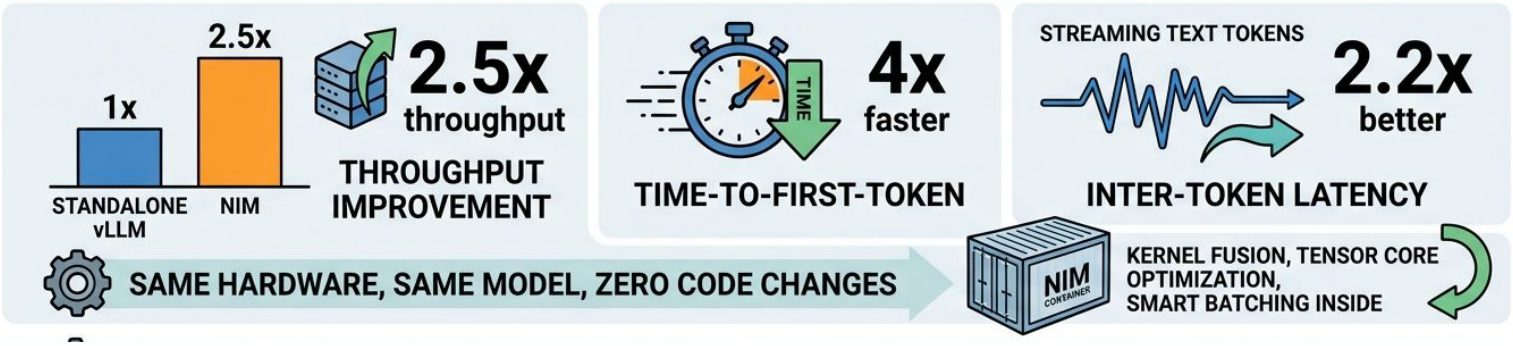
AI Community of Practice - March 2026

What NIM Actually Packages



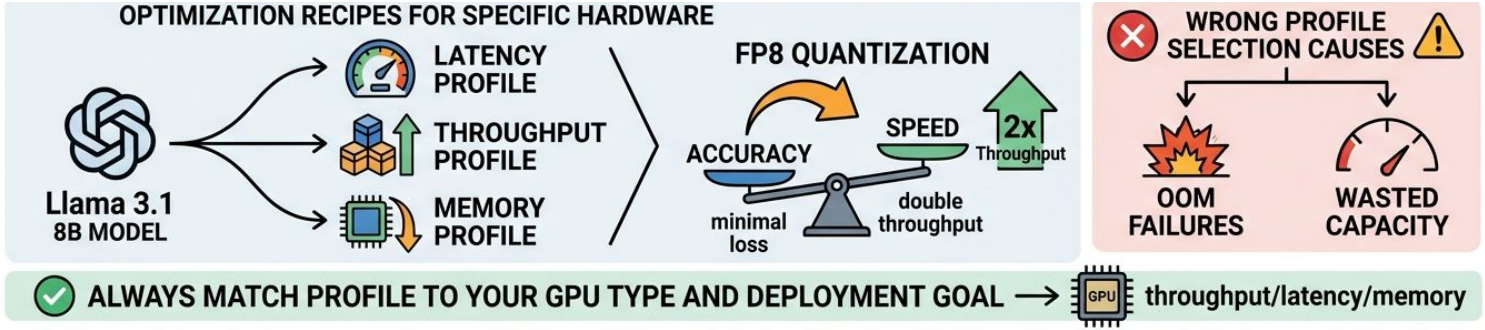
- Complete inference stack in one container image
- Optimized weights for Llama, Mistral, NVIDIA foundation models
- Pre-tuned engines: TensorRT-LLM, vLLM, SGLang
- OpenAI-compatible REST and gRPC APIs included
- All CUDA libraries and runtime dependencies bundled

Performance That Justifies the Hype



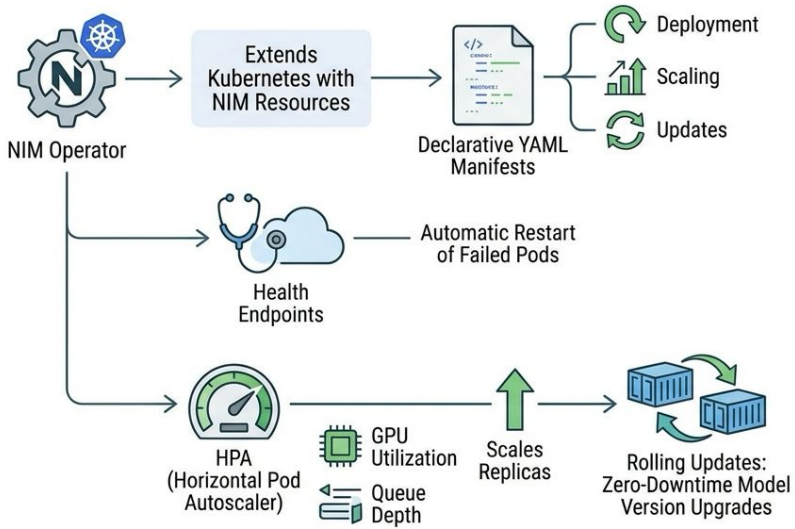
- 2.5x throughput improvement over standalone vLLM
- 4x faster time-to-first-token for responsive interactions
- 2.2x better inter-token latency during streaming
- Same hardware, same model, zero code changes required
- Kernel fusion, Tensor Core optimization, smart batching inside

The Profile System



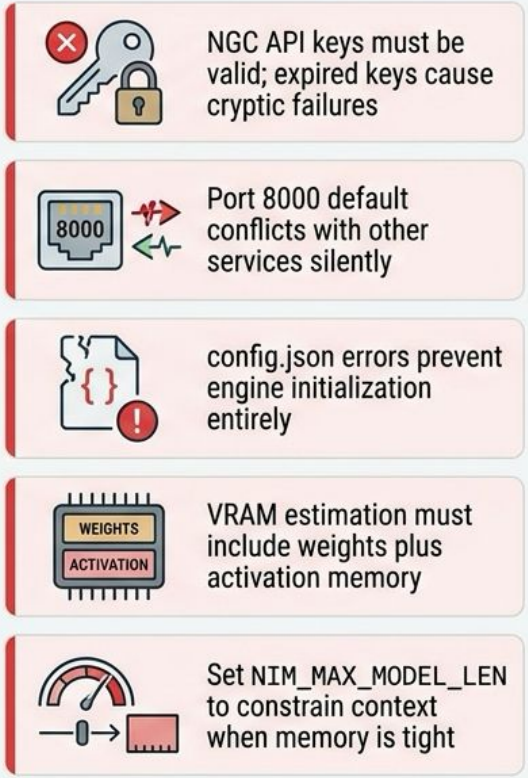
- Profiles provide hardware-specific model configurations
- Single model offers latency, throughput, and memory profiles
- FP8 quantization profiles double throughput with minimal accuracy loss
- Wrong profile selection causes OOM failures or wasted capacity
- Always match profile to your GPU type and deployment goal

NIM in Kubernetes -- From Containers to Platform



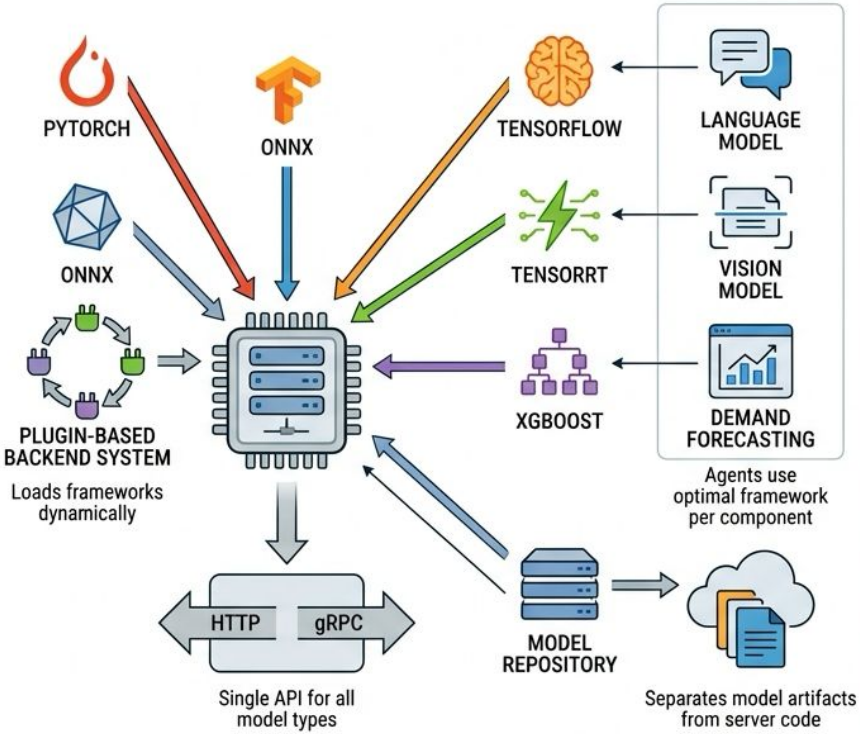
- NIM Operator extends Kubernetes with NIM-specific resources
- Declarative YAML manifests manage deployment, scaling, updates
- Health endpoints enable automatic restart of failed pods
- HPA scales replicas based on GPU utilization and queue depth
- Rolling updates achieve zero-downtime model version upgrades

NIM Production Pitfalls to Avoid



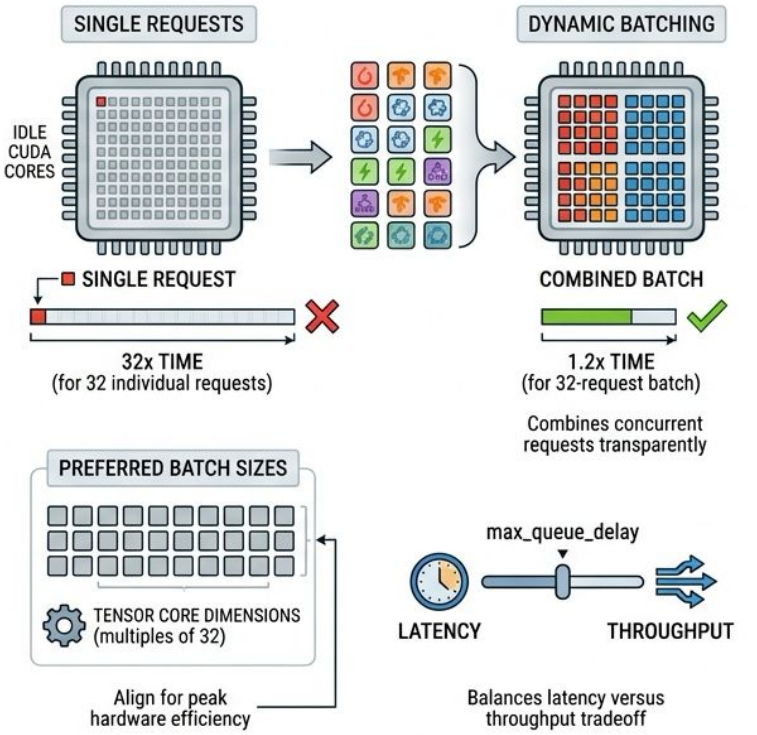
- NGC API keys must be valid; expired keys cause cryptic failures
- Port 8000 default conflicts with other services silently
- config.json errors prevent engine initialization entirely
- VRAM estimation must include weights plus activation memory
- Set NIM_MAX_MODEL_LEN to constrain context when memory is tight

Triton Inference Server



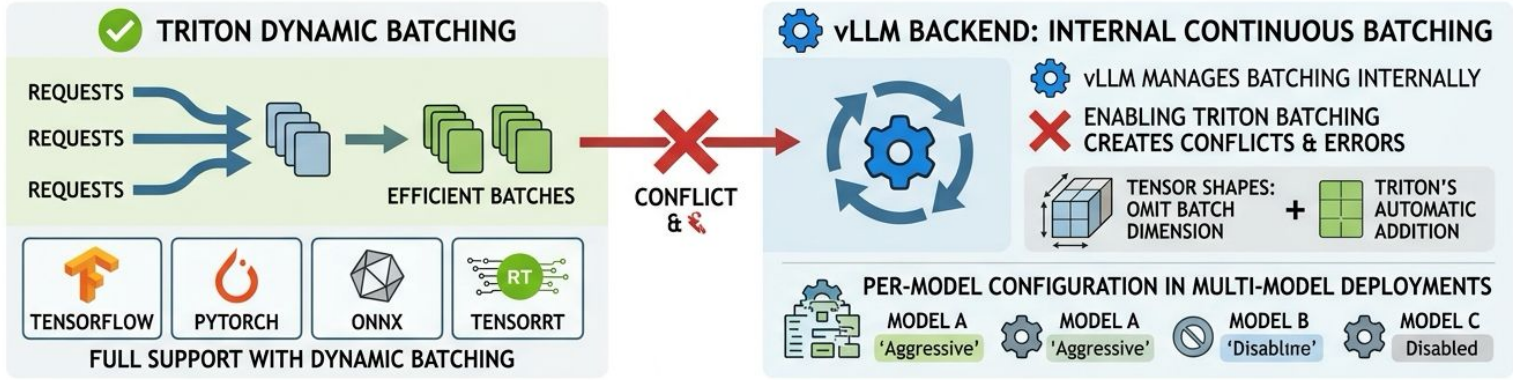
- Serves PyTorch, TensorFlow, ONNX, TensorRT, XGBoost models
- Plugin-based backend system loads frameworks dynamically
- Single API for all model types: HTTP and gRPC
- Agents use optimal framework per component, not one-size-fits-all
- Model repository separates model artifacts from server code

Dynamic Batching -- Triton's Throughput Multiplier



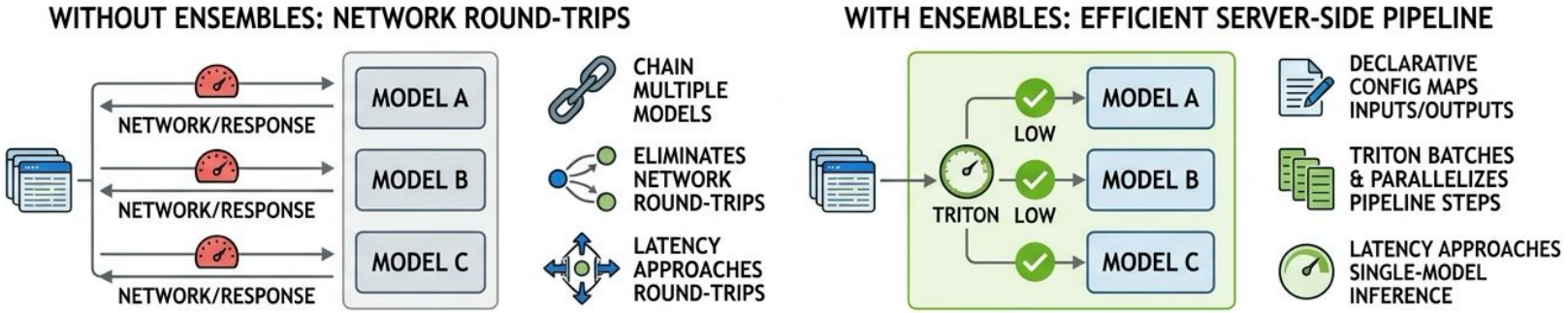
- Single requests waste thousands of idle CUDA cores
- Dynamic batching combines concurrent requests transparently
- 32-request batch runs in 1.2x time of single request, not 32x
- Preferred batch sizes align to Tensor Core dimensions (multiples of 32)
- max_queue_delay balances latency versus throughput tradeoff

Batching Gotcha -- Not All Backends Are Equal



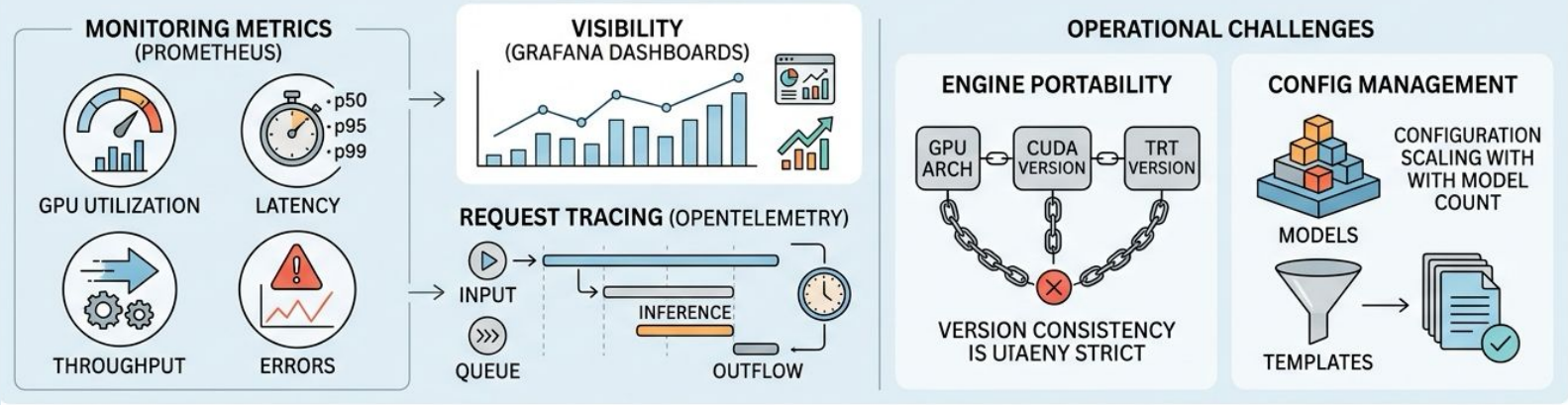
- TensorFlow, PyTorch, ONNX, TensorRT fully support dynamic batching
- vLLM backend implements its own continuous batching internally
- Enabling Triton batching for vLLM creates conflicts and errors
- Tensor shapes must omit batch dimension; Triton adds it automatically
- Per-model configuration required in multi-model deployments

Model Ensembles -- Pipelines Inside the Server



- Ensembles chain multiple models into server-side pipelines
- Eliminates network round-trips between pipeline stages
- Declarative config maps outputs of one model to inputs of next
- Triton batches and parallelizes steps within the pipeline
- Latency approaches single-model inference for multi-step workflows

Production Monitoring and Operational Realities



- Prometheus metrics cover GPU utilization, latency, throughput, errors
- Grafana dashboards surface health trends and anomalies at a glance
- OpenTelemetry traces reveal per-request execution breakdowns
- TensorRT engine portability requires strict version consistency
- Config complexity scales with model count; use templates to manage

Key Takeaways and Bridge to Chapter 4.6

- NIM delivers pre-optimized LLM inference with zero code changes
- Triton unifies multi-framework serving with dynamic batching
- Profile selection and batching config drive production performance
- Enterprise features: security hardening, SLA support, monitoring
- Next chapter: TensorRT-LLM optimization deep dive under the hood